Sommaire

1. Résumé du projet 3
2. Contexte et problématique 3
3. Jeu de données et protocole anti-fuite 4
4. Méthodes 4
5. Résultats 5
6. Architecture logicielle 8
7. Interface et expérience utilisateur 9
8. Qualité logicielle 11
9. Feuille de route du projet 11
10. Compétences démontrées 12
11. Limites assumées et perspectives 12
Références 13

1. Résumé du projet

PaluScan est un projet complet de vision par ordinateur appliqué à un enjeu de santé publique : déterminer, à partir d'une image microscopique de cellule sanguine, si celle-ci est infectée par *Plasmodium falciparum*. En un mois, le projet couvre l'intégralité de la chaîne de valeur d'un système d'apprentissage automatique : audit d'un corpus public de 27 558 cellules, conception d'un protocole d'évaluation à l'épreuve des fuites statistiques, entraînement d'un réseau convolutif léger sur simple processeur, statistique inférentielle écrite à la main, explicabilité par occlusion, export ONNX et déploiement d'une application web interactive de démonstration.

0,9914	96,8 %	95,4 %	< 10 ms
AUC ROC (test)	Sensibilité	Spécificité	Latence CPU

Les performances annoncées sont encadrées : intervalles de Wilson pour les proportions, bootstrap à 10 000 tirages stratifié par lame pour l'AUC, test de McNemar contre la baseline. Chaque prédiction du service web est accompagnée de sa carte d'occlusion, qui montre visuellement où le modèle a porté son attention. Le projet est versionné sous Git, testé (31 tests pytest), linté (ruff), épaulé par une CI GitHub Actions et livré avec un conteneur Docker prêt pour Hugging Face Spaces.

2. Contexte et problématique

Le paludisme demeure l'une des maladies infectieuses les plus meurtrières au monde. Le Cameroun, pays d'origine du contexte d'étude, figure parmi les quinze pays les plus touchés : l'Organisation mondiale de la santé y dénombre plus de trois millions de cas confirmés et environ onze mille décès par an (Rapport mondial sur le paludisme, OMS, 2024). Le diagnostic de référence reste la lecture microscopique du frottis sanguin : un technicien compte les hématies et cherche les parasites à l'œil nu, une tâche longue, fatigante, dépendante de l'expertise disponible et sujette aux variations inter-observateurs.

Les jeux de données publics de frottis photographiés offrent aujourd'hui une opportunité réelle : apprendre à un modèle de vision à repérer les signes d'infection au niveau cellulaire, pour assister le dépistage — prioriser les lames à relire, réduire la charge de travail, uniformiser la qualité. Ce projet ne prétend pas remplacer le biologiste ; il vise à démontrer la maîtrise du cycle complet de développement d'un modèle, de la donnée brute au service déployé, avec une exigence constante d'honnêteté statistique.

3. Jeu de données et protocole anti-fuite

Le corpus provient du jeu public NIH / LSHTM, distribué sous licence CC BY 4.0 : 27 558 images de cellules individuelles recadrées à 64 × 64 pixels depuis des frottis minces colorés au Giemsa, réparties équitablement entre cellules parasitées et saines. Chaque nom de fichier référence la lame d'origine — le corpus compte en réalité 1 408 lames distinctes, ce qui structure tout le protocole d'évaluation.

La fuite statistique est le piège majeur de ce jeu de données. Les cellules issues d'une même lame partagent le même patient, la même goutte de sang, le même réglage optique et le même terrain de coloration : elles sont quasi jumelles. Un découpage aléatoire image par image place donc des jumelles du jeu d'entraînement dans le jeu de test et gonfle artificiellement toutes les métriques — certaines réutilisations publiques du corpus annoncent ainsi plus de 99 % d'exactitude. PaluScan découpe **par lame** : aucune lame n'apparaît dans deux lots à la fois.

Lot	Lames	Cellules	Part des cellules
Entraînement	985	19 253	69,9 %
Validation	211	4 223	15,3 %
Test	212	4 082	14,8 %

Tableau 1 — Répartition des lots par lame (découpage groupé, semé)

Une baseline classique (analyse en composantes principales à 50 composantes suivie d'une régression logistique), évaluée sur le même protocole groupé, plafonne à une AUC de 0,7413 : elle fixe le plancher et prouve que le protocole ne sur-prime pas l'information mais supprime la fuite.

4. Méthodes

4.1 MalariaNet, un réseau résolument léger

MalariaNet empile trois blocs convolutifs (32, 64 puis 128 canaux), chacun suivi d'une normalisation par lot et d'un MaxPool, puis d'une tête dense avec décrochage. Le réseau totalise environ 62 000 paramètres — deux mille fois moins qu'un ResNet-50 — et s'entraîne en quelques minutes sur deux cœurs CPU. Ce choix est délibéré : dans un contexte de terrain, un modèle léger est un modèle déployable ; le projet démontre que 62 000 paramètres bien entraînés suffisent largement pour cette tâche binaire.

Hyperparamètre	Valeur
Optimiseur	Adam, taux d'apprentissage 0,001
Lot (batch)	64 images
Augmentation	rotations, miroirs, perturbations de luminosité
Arrêt anticipé	patience de 4 époques sur la perte de validation
Durée effective	16 époques (arrêt anticipé déclenché ensuite)
Matériel	CPU 2 cœurs, aucun GPU nécessaire

Tableau 2 — Configuration d'entraînement retenue

4.2 Une statistique écrite à la main

Toute la couche d'inférence statistique tient dans un unique module NumPy de deux cents lignes, sans dépendance à scipy ou statsmodels : courbes ROC et précision–rappel exactes, intervalles de Wilson (robustes aux petits effectifs, contrairement à l'approximation normale), bootstrap à 10 000 rééchantillonnages stratifié par lame pour l'AUC, seuil de Youden calculé sur la validation, test exact de McNemar pour comparer deux modèles appariés, courbe de calibration et score ECE. Ce choix pédagogique et technique a un double mérite : chaque ligne du calcul est auditable, et la suite de tests couvre les cas limites (intervalle dégénéré, zéro discordance).

4.3 Explicabilité par occlusion

Pour chaque prédiction, une fenêtre de 8 × 8 pixels parcourt l'image avec un pas de 4 ; à chaque position, on mesure la chute du logit de la classe prédite quand la fenêtre est masquée. Les régions dont le masquage détruit la confiance portent l'information discriminante. La carte obtenue, normalisée puis superposée en dégradé rouge-jaune, permet de vérifier d'un coup d'œil que le modèle regarde le parasite — et non un artefact de coloration. La méthode n'exige aucun gradient et fonctionne donc directement sur le moteur d'inférence ONNX.

5. Résultats

Métrique	Valeur	IC 95 %
AUC ROC	0,9914	[0,989 ; 0,994]
Sensibilité	96,8 %	[96,0 % ; 97,5 %]
Spécificité	95,4 %	[94,5 % ; 96,3 %]
Exactitude	96,0 %	[95,3 % ; 96,6 %]
Précision moyenne (AP)	0,9915	s. o.

Tableau 3 — Performances sur le jeu de test (4 082 cellules, 212 lames jamais vues)

Le test de McNemar apparié confirme l'écart avec la baseline : sur les 1 348 cellules tranchées différemment entre les deux modèles, la baseline se trompe seule 1 255 fois contre 93 pour le CNN ($p < 0{,}001$). L'écart n'est pas un artefact d'échantillonnage.

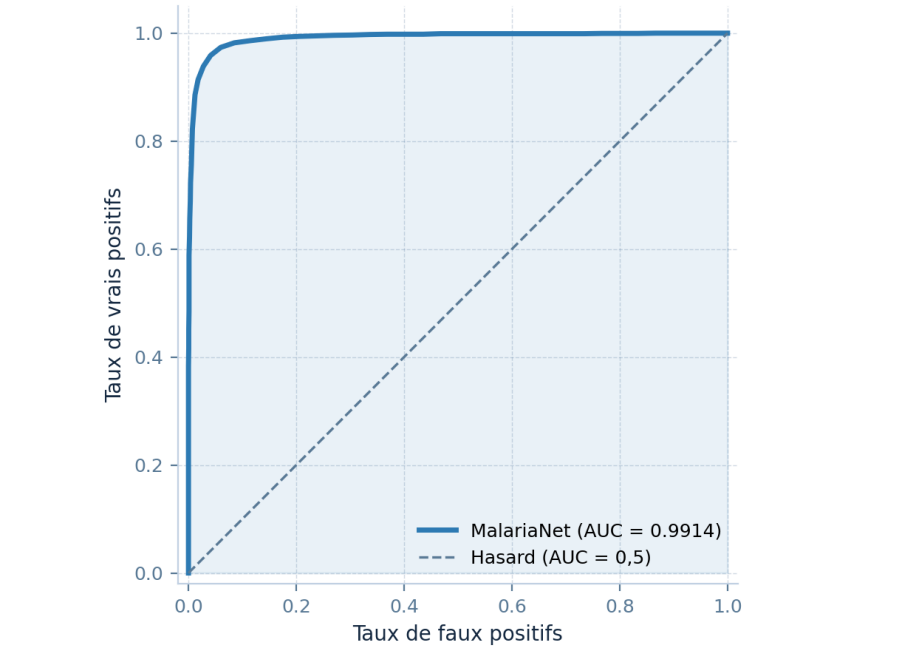


Figure 1 — Courbe ROC sur le jeu de test ; la diagonale en pointillés représente le hasard.

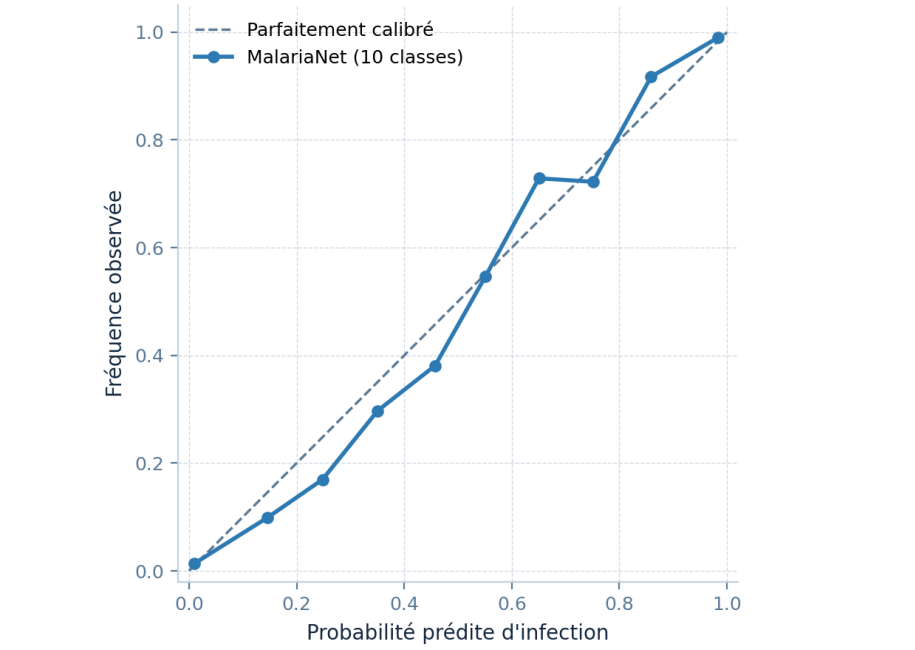


Figure 2 — Calibration : probabilité prédite contre fréquence observée, dix classes d'équiprobabilité.

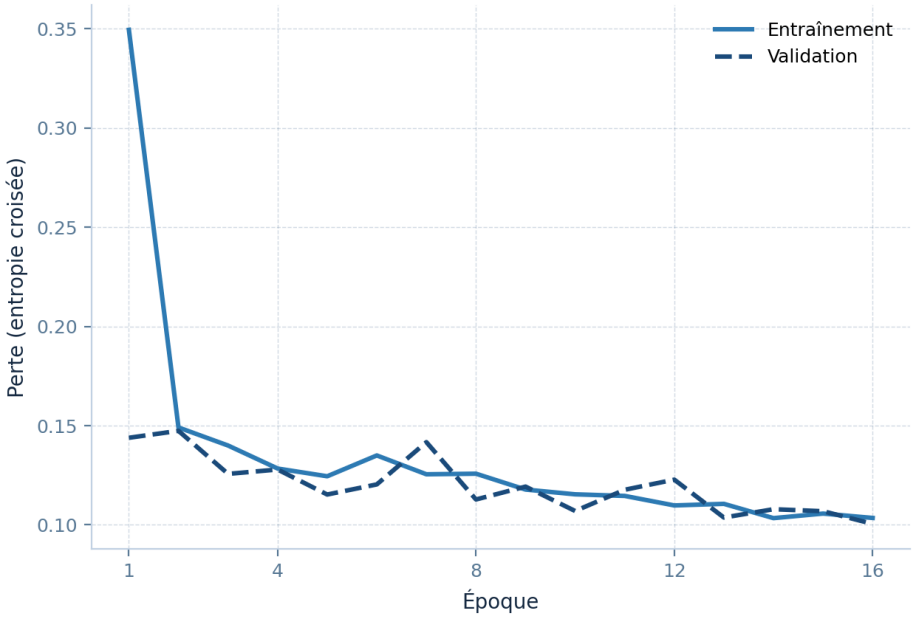


Figure 3 — Perte d'entraînement et de validation par époque : convergence stable, pas de sur-apprentissage.

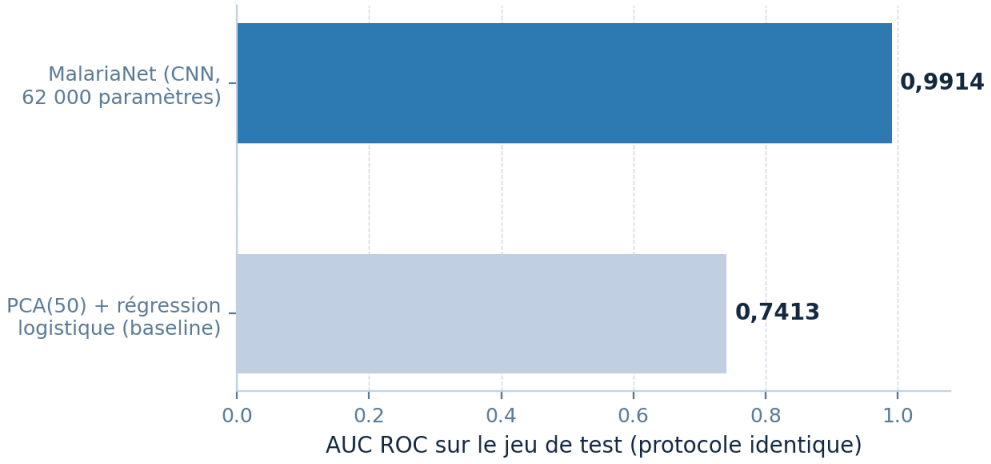


Figure 4 — Comparaison de l'AUC sur le même protocole groupé par lame.

La courbe de calibration montre un modèle globalement bien calibré dans les zones de forte densité (les classes extrêmes rassemblent 87 % des cellules de test), avec une légère sur-confiance dans les classes médianes — les scores intermédiaires restent par construction rares sur un jeu équilibré et recadré. C'est précisément pour ces zones d'incertitude que l'application expose le seuil de décision de Youden plutôt qu'un seuil arbitraire de 0,5.

6. Architecture logicielle

Le code est organisé en bibliothèque installable (*src/*, gestionnaire uv, Python 3.12) séparée de l'application web. Les dépendances sont cloisonnées par groupes : PyTorch n'est requis qu'à l'entraînement, le conteneur de démonstration n'embarque que FastHTML, NumPy, Pillow et ONNXRuntime — soit un livrable de quelques dizaines de mégaoctets, sans GPU.

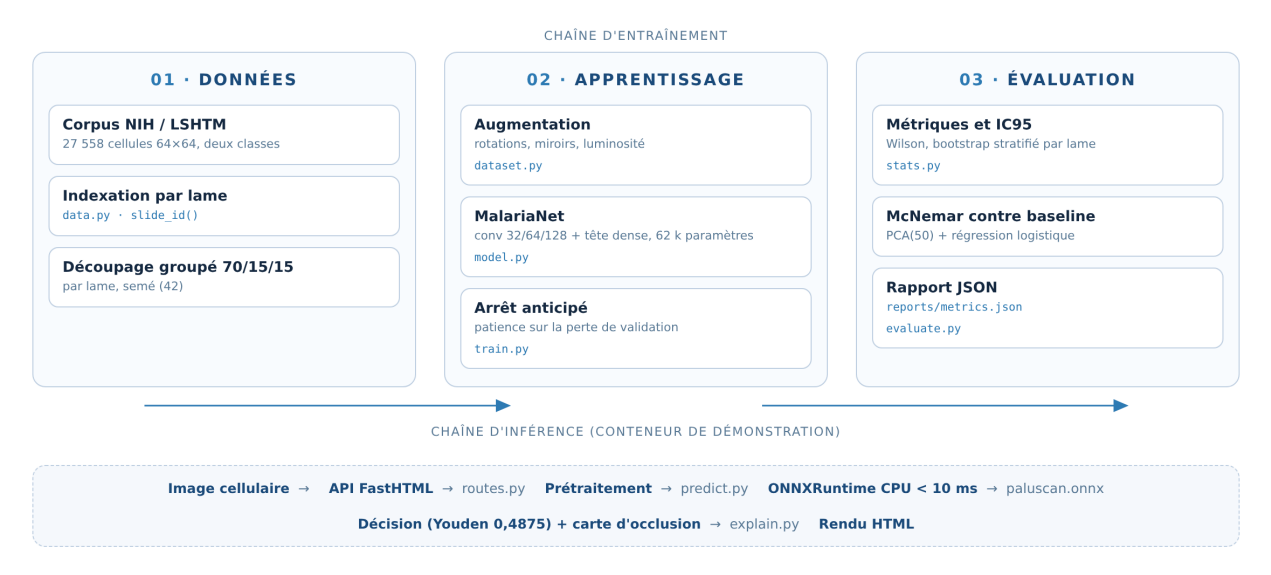


Figure 5 — Architecture : chaîne d'entraînement (haut) et chaîne d'inférence de la démonstration (bas).

Module	Rôle
data.py	découverte du corpus, extraction de la lame, découpage groupé 70/15/15
dataset.py	chargement, normalisation, augmentation géométrique et photométrique
model.py	MalariaNet (PyTorch), 62 000 paramètres
train.py	boucle d'entraînement, arrêt anticipé, reprise depuis un point de contrôle
stats.py	Wilson, ROC/PR, bootstrap, McNemar exact, calibration, Youden
evaluate.py	évaluation du test, courbes, rapport metrics.json
explain.py	cartes d'occlusion via ONNXRuntime
predict.py	encapsulation d'inférence (session unique, latence mesurée)
app/	application FastHTML : vues, routes API, statiques (CSS/JS embarqués)

Tableau 4 — Modules de la bibliothèque et responsabilités

7. Interface et expérience utilisateur

La démonstration web est conçue comme la vitrine du projet : thème sombre de laboratoire, typographie éditoriale (Fraunces / Inter), animations sobres révélées au défilement, inclinaison 3D des cartes au pointeur, jauge animée et squelettes de chargement. L'interface respecte les préférences système (mode contraste réduit, mouvement réduit) et reste entièrement fonctionnelle au clavier. Trois écrans structurent le parcours : accueil, laboratoire d'analyse, méthode.



Figure 6 — Page d'accueil : héros animé évoquant le champ microscopique, chiffres clés en compteurs.

Le laboratoire accepte le glisser-déposer d'une à douze images, propose neuf exemples du jeu de test et affiche pour chaque cellule : la décision au seuil de Youden, la jauge de probabilité d'infection animée, la latence mesurée côté serveur, la superposition de la carte d'occlusion activable d'un geste, et un curseur de prévalence qui recalcule en direct les valeurs prédictives positive et négative par la règle de Bayes — une façon concrète de montrer qu'un même modèle n'a pas la même valeur selon le contexte épidémiologique.

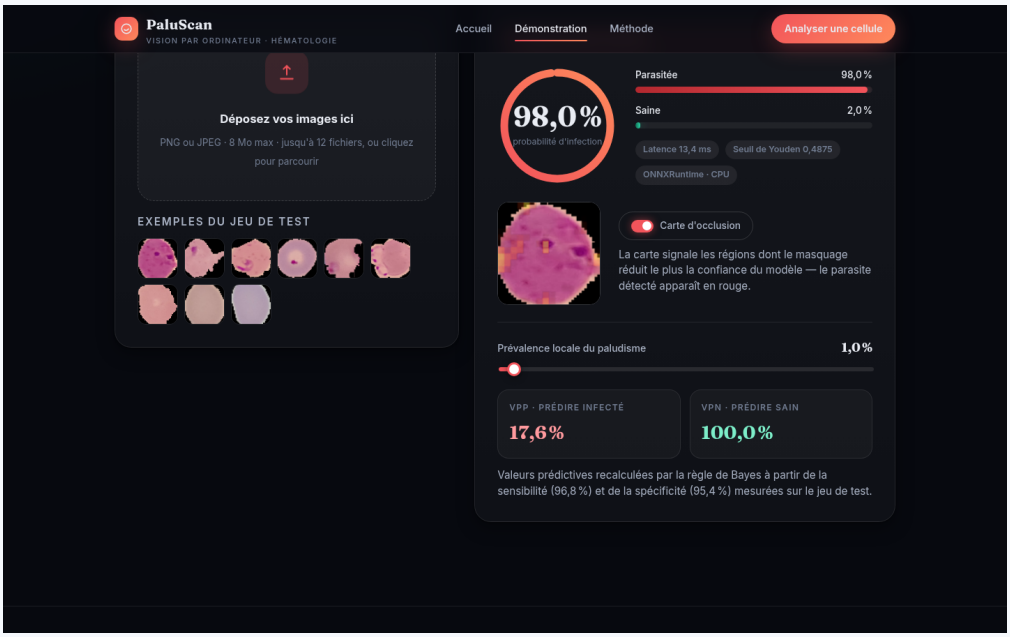


Figure 7 — Résultat d'analyse : verdict, jauge animée, cartes de probabilité, occlusion, valeurs prédictives.

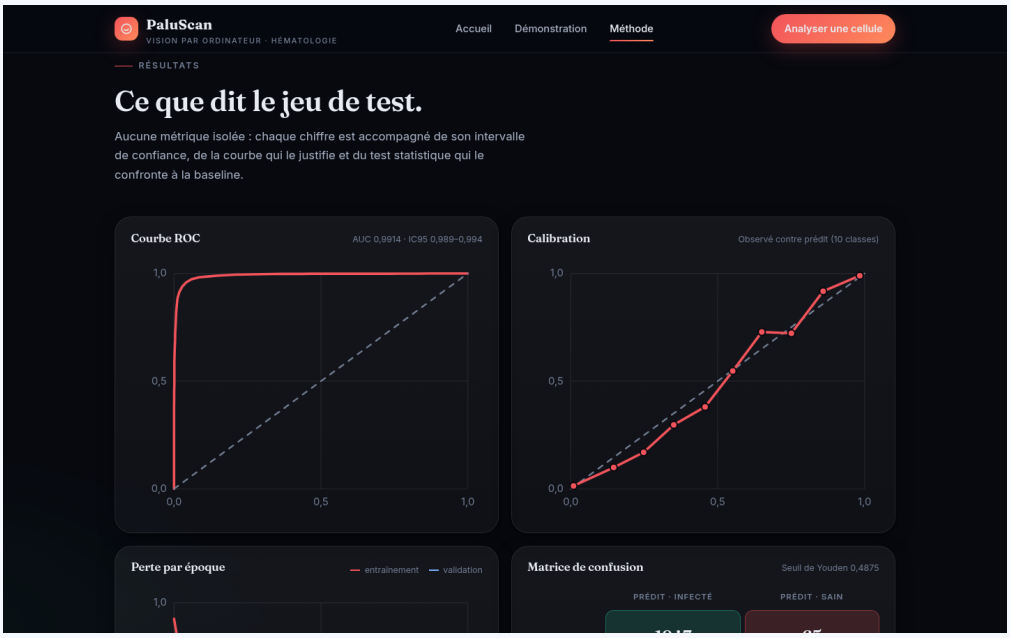


Figure 8 — Tableau de bord de la page méthode : ROC, calibration et matrice de confusion rendus en SVG.

Toutes les figures de la page méthode sont des SVG générés côté serveur à partir de metrics.json : le déploiement n'embarque aucune bibliothèque de tracé côté client, les courbes s'animent au tracé (stroke-dashoffset) et restent nettes à toute résolution.

8. Qualité logicielle

Le projet est versionné sous Git avec une historique de commits propres et descriptifs. La configuration qualité comprend : 31 tests pytest couvrant la statistique (cas limites inclus), le découpage groupé (absence de fuite vérifiée par test), l'inférence ONNX et l'application web (pages, API, gestion d'erreurs) ; ruff en linter et formateur, configuré py312 ; une chaîne GitHub Actions qui rejoue lint, format et tests à chaque poussée ; et un conteneur Docker reproductible (verrou uv.lock), compatible Hugging Face Spaces sur le port 7860.

Contrôle	Portée
pytest — 31 tests	statistique, découpage sans fuite, inférence, API web (succès et erreurs)
ruff (lint + format)	zéro avertissement sur l'ensemble des modules Python
GitHub Actions	lint, format, tests à chaque poussée sur la branche principale
Docker + uv.lock	image reproductible, port 7860, prête pour Hugging Face Spaces

Tableau 5 — Vérifications automatisées

9. Feuille de route du projet

Le projet a été conduit en un mois selon le plan suivant ; les quatre premières étapes sont réalisées et livrées.

Étape	Contenu	Statut
Semaine 1	cadrage, audit du corpus NIH, découpage par lame, script de préparation reproductible	Réalisé
Semaine 2	baseline PCA + régression logistique, module statistique complet, tests	Réalisé
Semaine 3	MalariaNet, augmentation, entraînement avec arrêt anticipé, export ONNX	Réalisé
Semaine 4	évaluation, explicabilité, application FastHTML, conteneur, documentation	Réalisé
Suite — master 1	validation externe sur un corpus d'un autre microscope, mesure inter-observateur, publication du Space	Envisagé
Suite — master 2	détection sur image entière (segmentation), fusion multi-lames, étude clinique avec partenariat local	Envisagé

Tableau 6 — Plan de réalisation sur un mois et suite envisagée

10. Compétences démontrées

Le projet a été calibré pour couvrir explicitement les attendus d'un master en apprentissage automatique : chaque compétence revendiquée ci-dessous est prouvée par un livrable précis du dépôt.

Domaine	Preuve concrète
Algorithmique et programmation	ROC/PR exactes, McNemar exact, bootstrap et Wilson écrits en NumPy pur (stats.py)
Probabilités et statistique	intervalles de confiance, calibration, seuil de Youden, test apparié, règle de Bayes exposée dans l'UI
Apprentissage profond	conception et entraînement d'un CNN, augmentation, arrêt anticipé, comparaison à une baseline
Ingénierie des données	audit du corpus, découpage groupé anti-fuite, scripts reproductibles semés
Génie logiciel	architecture en paquet, 31 tests, lint zéro défaut, CI, conteneur, historique Git propre
Production et explicabilité	export ONNX, latence mesurée, cartes d'occlusion, interface web premium déployable

Tableau 7 — Compétences et preuves associées dans le dépôt

11. Limites assumées et perspectives

Un dossier d'honnêteté scientifique doit dire ce que le modèle ne sait pas. Le corpus ne contient que des cellules déjà recadrées : en conditions réelles, une étape de segmentation de l'échantillon serait nécessaire en amont. Quelques images étiquetées « saines » sont en réalité des globules blancs, artefact documenté du corpus d'origine, qui dégrade légèrement la spécificité mesurée. Le modèle a été entraîné sur un seul terrain de coloration et un seul parc de microscopes : une généralisation à d'autres plateformes exigerait une validation externe systématique. Enfin, PaluScan est un outil d'étude et d'assistance : la lecture d'un frottis reste l'affaire d'un biologiste.

Ces limites dessinent les perspectives naturelles du travail : validation externe multi-site, détection complète sur image de frottis entière (segmentation puis comptage), étude des inter-observateurs assistés contre non assistés, et mesure d'impact réel sur la charge de lecture d'un laboratoire de terrain.

Références

World Health Organization. World Malaria Report 2024. Genève : OMS, 2024.

Rajaraman, S., Antani, S. K., et al. Pre-trained convolutional neural networks as feature extractors toward improved malaria parasite detection in thin blood smear images. PeerJ, 6 : e4568, 2018.

Silva, D., G de Oliveira, L., et al. Towards automatic detection of malaria parasites in blood smears : dataset (NIH / LSHTM Malaria Cell Images). Mendeley Data, V1, doi : 10.17632/y7z2vg7fmy.1, 2019.

Wilson, E. B. Probable inference, the law of succession, and statistical inference. Journal of the American Statistical Association, 22(158) : 209–212, 1927.

McNemar, Q. Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika, 12(2) : 153–157, 1947.

Zeiler, M. D., Fergus, R. Visualizing and understanding convolutional networks. In ECCV, p. 818–833, 2014.